

Distributed Computation of Skyline Probability over Uncertain Preferences

Easwar Kilari

NIT Warangal

Warangal, Telangana, India

Microsoft Corporation India Pvt Ltd

Hyderabad, India

easwarkilari@gmail.com

Sriram Kailasam

NIT Warangal

Warangal, Telangana, India

sriramkailasam@gmail.com

Abstract

Computing skyline probabilities in the presence of uncertain preferences is a $\#P$ complete problem. The current state-of-the-art, the *Usky-base* algorithm, operates sequentially and, despite its ability to prune significant portions of joint probabilities, suffers from an exponentially increasing lattice size relative to the database size. Consequently, even small datasets result in prohibitively long computation times. The *Usky-base* algorithm employs a prefix-based level wise absorption algorithm that is not amenable to parallelization. We reformulated the *Usky-base* algorithm as an enumeration problem within a combinatorial tree search paradigm. Then, we used the DiGTrees API, an open-source framework for combinatorial tree search to express the skyline computation. We performed comprehensive experiments on a distributed environment with a fixed number of workers and with elastic scaling. We observed that the proposed system scales well on both real and synthetic datasets.

CCS Concepts

• **Computing methodologies** → **Distributed algorithms**; *Discrete space search*.

Keywords

skyline probability, uncertain preferences, distributed framework, combinatorial search

ACM Reference Format:

Easwar Kilari and Sriram Kailasam. 2025. Distributed Computation of Skyline Probability over Uncertain Preferences. In *26th International Conference on Distributed Computing and Networking (ICDCN 2025)*, January 04–07, 2025, Hyderabad, India. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3700838.3700859>

1 Introduction

An object p in a multi-dimensional dataset is said to dominate another object q iff the object p is at least as good as q in all dimensions and better than q in at least one dimension. A skyline object is one which is not dominated by any other object in the

dataset. The set of skyline objects represent the best options available in a multi-dimensional dataset and is widely used in multi-criteria decision-making, data cleaning, recommendation systems, etc. Many techniques such as R-tree [9], B-tree [17], space-filling curves [11], etc., have been designed to compute skyline objects efficiently.

In 2007, Pei et al. [12] first studied the skyline computation problem for uncertain data wherein the attribute's values are uncertain, i.e. its values are drawn from a particular probability distribution. For example, data generated by sensor networks falls under this category. Here, an object can be considered a skyline object with some probability. The traditional skyline definitions were extended to the probabilistic setting wherein an object belongs to the skyline if its skyline probability is greater than a specified threshold. Thereafter, many techniques for efficient computation of skyline probabilities for uncertain data have been proposed [4, 14].

Another notion of uncertainty was studied by Sacharidis et al. [15] wherein the attribute values are fixed but the user preferences are uncertain. Consider a categorical attribute (say) type of food served by a restaurant; it can take different values such as South-indian, North-indian, Chinese, Mexican, etc. A user may prefer one type of cuisine over another on a particular day, i.e. his preference is contextual. In these situations, it is hard to model the user's preferences as strict partial orders. Rather, based on historical data, we may be able to specify the probability that a user prefers one value over another for a given categorical attribute. Considering the user's preference probabilities on all attributes, we can then calculate the probability that the user prefers one restaurant over another. The skyline probability of a particular restaurant would be the probability that no other restaurant is preferred to this one on all attributes. Zhang et al. [18] showed that computing the skyline probability with uncertain preferences is $\#P$ complete. They also illustrated that the independent object dominance assumption made while computing skyline probabilities for uncertain data is invalid for uncertain preferences. Thus, the skyline computation techniques proposed for uncertain data are not applicable for uncertain preferences. In this paper, we investigate this problem of efficiently computing the skyline probability of an object over uncertain preferences.

Zhang et al. [18] proposed the first deterministic, sequential algorithm for computing the skyline probability of an object over uncertain preferences. Their algorithm computes the joint probability of every element in the power set of attribute values. This algorithm does not scale even to a database of moderate size as it contains an exponential number of terms. Consequently, they



This work is licensed under a Creative Commons Attribution International 4.0 License.

ICDCN 2025, January 04–07, 2025, Hyderabad, India

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1062-9/25/01

<https://doi.org/10.1145/3700838.3700859>

proposed an approximation algorithm based on Monte Carlo estimation to improve performance. Pujari et al. [13] made an interesting observation that under certain conditions the aggregate of a portion of the terms in the skyline probability expression becomes zero. Thus, if such sets (a.k.a. zero contributing sets) can be identified in advance, then there is no need to compute the probabilities of those terms. They empirically showed that major portions (almost 99.99%) of the power-set lattice for real-world databases are pruned with this approach. Their proposed algorithm *Usky-base* uses a prefix-based absorption technique to eliminate the zero-contributing sets. With this approach, they were able to compute the exact skyline probabilities of objects and scale to moderate size databases. However, even their algorithm is sequential and as shown in the experiments section does not scale to large datasets. Zhu et al. [19] proposed a parallel algorithm to speedup the *Usky-base* algorithm. However, it suffers from severe load-imbalance due to pruning of the zero-contributing sets across the workers (see Section 4.4 for the experiment results).

In this paper, we reformulate the *Usky-base* algorithm as an enumeration problem within a combinatorial tree search paradigm. This enables us to use any of the frameworks supporting combinatorial tree search to implement our algorithm. We surveyed several open-source frameworks w.r.t. following features: easy-to-use API for expressing combinatorial search, ability to exploit parallelism within and across machines (support for parallel and distributed computing), ability to handle worker failures during runtime, support for dynamic load-balancing and elastic scaling. We found that the DiGTreeS [7] framework provides support for all the above features whereas others lack one or more of these features. Hence, we implemented our algorithm for computing skyline probabilities using the DiGTreeS framework. We performed comprehensive experiments on a distributed environment with a fixed number of workers and with elastic scaling. We observed that our proposed system scales well on both real and synthetic datasets with parallel execution efficiency of 78% on 128 workers for the largest dataset.

This paper is organized as follows: Section 2 explains the relevant background for computing skyline probabilities over uncertain preferences and the related work; Section 3 discusses our algorithm design and its implementation; Section 4 describes the experimental results; conclusions and future work are presented in Section 5.

2 Background and Related Work

2.1 Skyline probability over uncertain preferences

Borrowing notations from Zhang et al. [18], we illustrate skyline probability computation over uncertain preferences with an example. Table 1 shows 3 objects along with their attribute values for dimensions A and B. Let $<$ denote the dominance relationship between objects or values, e.g. $a1 < a2$ means that $a1$ is preferred over $a2$. Let the corresponding probability be $Pr(a1 < a2)$. To denote that the preference of $a1$ is greater than or equal to $a2$, we use $a1 \leq a2$. With these notations in place,

$$Pr(a1 \leq a2) = \begin{cases} 1, & a1 = a2 \\ Pr(a1 < a2), & a1 \neq a2 \end{cases}$$

Assuming that the attribute values are distinct and all the pair-wise

Table 1: 2-dimensional dataset with 3 objects

Objects	A	B	
O	a1	b1	$Pr(a1 < a2) = 0.5,$
Q^1	a2	b1	$Pr(a2 < a1) = 0.5,$
Q^2	a2	b2	$Pr(b1 < b2) = 0.5,$
			$Pr(b2 < b1) = 0.5$

preference probabilities are 0.5 as shown in Table 1, we can compute the skyline probabilities as follows: let e_i be the event that object Q^i is preferred over O i.e. $(Q^i < O)$; then $sky(O)$ is the probability that none of the objects Q^i dominate O . Thus,

$$sky(O) = Pr\left(\bigcap_{i=1}^n \bar{e}_i\right) = 1 - Pr\left(\bigcup_{i=1}^n e_i\right) \quad (1)$$

Let θ denotes the set of all objects, $E_J = \bigcap_{Q^i \in J} e_i$ for a given $J \in 2^\theta$.

Then, by using the inclusion-exclusion principle,

$$sky(O) = 1 + \sum_{k=1}^n (-1)^k \sum_{J \in 2^\theta, |J|=k} Pr(E_J) \quad (2)$$

For our example, $sky(O) = 1 - Pr(e_1) - Pr(e_2) + Pr(e_1 \cap e_2)$.

Assuming mutual independence between the preferences of the different dimensions [18],

$$Pr(E_J) = \prod_{j=1}^d \prod_{v \in \text{distinct}(O; J, j)} Pr(v \leq O_j) \quad (3)$$

where d is the number of dimensions in the dataset and $\text{distinct}(O; J, j)$ is the set of all distinct attribute values of dimension j in the objects in J excluding the attribute values of O . For our example, $Pr(e_1) = Pr(b2 \leq b1) = 0.5$, $Pr(e_2) = Pr(a2 \leq a1) \cdot Pr(b2 \leq b1) = 0.25$, $Pr(e_1 \cap e_2) = Pr(a2 \leq a1) \cdot Pr(b2 \leq b1) = 0.25$. Thus, $sky(O) = 0.5$.

Brute-force computation of the skyline probability of an object requires traversal of the entire power-set lattice whose size is 2^θ where θ = total number of objects, and computing the joint-probability of every element. Zhang et al. [18] showed that this problem is #P complete. Thus, this approach does not scale to even moderate-sized databases.

2.2 Related Work

In probabilistic skyline computation on uncertain data, an object is represented as a set of instances according to some probability distribution. The existing works [2, 4, 12, 14] assume that an instance of an object does not depend on the instances of other objects. This independent instance domination allows these works to use transitivity among the dominances to improve the efficiency of skyline computation. However, this independent object dominance is shown to be invalid when uncertain preferences are considered [18]. Thus, the skyline computation techniques derived for uncertain data are not applicable for uncertain preferences.

Zhang et al. [18] proposed the first sequential deterministic algorithm to compute skyline probability of an object in a multi-dimensional space with uncertain preferences. However, they found that their sequential algorithm was not scalable as it has to navigate a search space (comprising the power set lattice) containing an exponential number of terms. To overcome these performance

issues, the authors proposed an approximation algorithm based on Monte Carlo estimation. Pujari et al.[13] made an interesting observation that if the zero-contributing sets could be identified in advance, then major portions of the power-set lattice can be pruned. Thereby, the skyline probabilities can be computed more efficiently. They proposed a prefix-based absorption technique called *Usky-base* algorithm to eliminate the zero-contributing sets from the power-set lattice. They empirically showed that major portions are pruned with this approach. Even with these improvements, the *Usky-base* algorithm does not scale to large datasets. Zhu et al.[19] proposed a parallel algorithm for uncertain preferences based on the *Usky-base* algorithm. They partitioned the power-set lattice into as many independent regions as the number of workers and assigned one to each worker. Each worker then runs the *Usky-base* algorithm on its partition. However, in this approach, the load distribution across the workers is highly uneven due to pruning of the zero-contributing sets.

Parallel computation of skyline probabilities for uncertain data are proposed for different architectures such as CPUs and GPUs [3]. Certain MapReduce based algorithms have also been proposed for uncertain data [5]. However as discussed earlier, these techniques are not applicable to uncertain preferences.

3 System Design

3.1 Problem statement

Given a set of objects in a multidimensional dataset with uncertain preferences, the objective of this paper is to design and implement a scalable algorithm to compute the skyline probability of a given object O in this set efficiently.

3.2 Overview of *Usky-base* algorithm

Pujari et al. [13] introduced the notion of zero-contributing sets to refer to portions of the power-set lattice that do not contribute to the skyline probability, i.e. the sum of those terms (considering the +ve or -ve sign of the term) in the skyline probability expression is zero. They proposed the *Usky-base* algorithm that uses the prefix-based k -level absorption technique to eradicate the zero-contributing sets.

Definition 1. *k -level absorption from [13]:* For any non-empty set of objects S in the multi-dimensional dataset where $|S| = k - 1$, if $\text{distinct}(O; S \cup \{y\}) = \text{distinct}(O; S \cup \{x, y\})$ for $x, y \notin S$, then $S \cup \{x\}$ absorbs $S \cup \{y\}$ at level k of the lattice. $\text{distinct}(O; S)$ is defined as the union of the distinct attribute values of S for each dimension w.r.t. O .

Fig. 1 is an illustration of k -level absorption algorithm for the sample dataset given in Table 2. Notice that each term in the figure denotes a joint probability computation and k denotes the level from the bottom of the lattice. For $k = 0$, we have the entire power-set lattice. For $k = 1$, the candidates are the singleton elements ($\{1\}, \{2\}, \{3\}, \{5\}, \{4\}$). To evaluate if there are any absorbed terms at level 1, we first compute $\text{distinct}(O; \{x\})$ for all singleton elements $\{x\}$. Then we compute $\text{distinct}(O; \{x, y\})$ by pairing x with every other object y . In this example, $\text{distinct}(O; \{4\}) = \{b2, c2, d2\}$ which is the same as $\text{distinct}(O; \{1, 4\})$. Hence, object Q^1 absorbs Q^4 as per Definition 1. And therefore the entire portion of the lattice

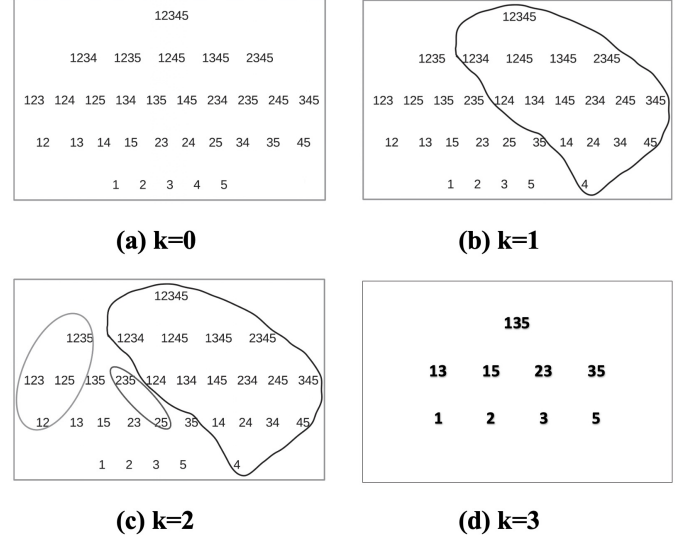


Figure 1: k -level prefix absorption in the power-set lattice

containing 4 is a zero-contributing set (marked in Fig. 1b) and can be eliminated. For the next-level $k = 2$, we consider 2-element candidate subsets that do not include Q^4 . Repeating the same procedure as before, we see that $\{1, 3\}$ absorbs $\{1, 2\}$ as $\text{distinct}(O; \{1, 2\})$ is same as $\text{distinct}(O; \{1, 2, 3\})$. Similarly, $\{2, 3\}$ absorbs $\{2, 5\}$. Thus, the corresponding portions of the power-set lattice are eliminated. The algorithm proceeds level-wise and terminates with 9 elements as shown in Fig. 1d. Pujari et al.[13] show empirically that only a very small portion of the exponential search space remains after prefix-based absorption is applied at each level. However, despite its efficiency, the *Usky-base* algorithm operates sequentially, and takes enormous time for large datasets (see last column of Table 4 for sequential execution time).

Table 2: Running example.

Objects	A	B	C	D
O	a1	b1	c1	d1
Q^1	a1	b2	c1	d2
Q^2	a2	b1	c2	d1
Q^3	a2	b1	c1	d2
Q^4	a1	b2	c2	d2
Q^5	a1	b1	c2	d2

3.3 Key insight for parallelizing the *Usky-base* algorithm

We can reformulate the exploration of the power-set lattice as shown in Fig. 2a. A node in the power-set lattice at a given level can be generated from a node at the previous level by adding a single object to it. If we order the objects as per their identifiers and impose a rule that the next-level children of a particular node can be generated only by adding objects greater than the last-added

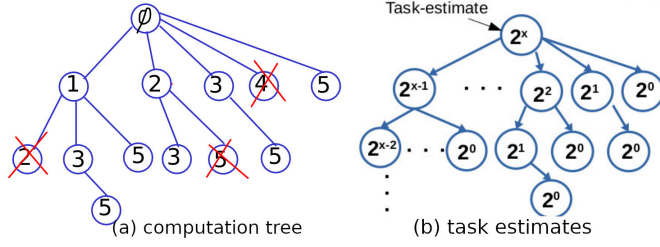


Figure 2: Exploration of power-set lattice for distributed skyline computation

object (marker), then each node will be generated only once. To realize this, consider the exploration from the empty-object set. The marker is initialized to 0. We add objects to it (that are greater than the marker) one at a time and generate the first-level candidate sets. The second-level candidate sets generated according to this rule are unique. Thus, the entire power lattice can be explored without redundant generation of candidate sets.

According to the *Usky-base* algorithm, a node at a particular level can be absorbed by another node at the same level. If a node gets absorbed, then the subsequent portion of the lattice including the absorbed node is a zero-contributing set and can be eliminated. To achieve this, we associate with each node an object-set called markers that contains objects having identifiers greater than the marker. During exploration, we generate the child nodes by adding an object from the markers one at a time. Next, we evaluate whether any of the child nodes generated are redundant (i.e. absorbed by another child node). If that happens, we eliminate these redundant nodes from the markers. A copy of the updated markers is passed on to the generated child nodes for further exploration. This ensures that the zero-contributing sets are avoided. Further, it allows independent exploration of the subtree below each child node. This insight allows us to parallelize the *Usky-base* algorithm.

It is important to note however, that the exploration tree in Fig. 2b is highly skewed, where the left-most sub-tree can have a maximum of (2^{x-1}) nodes, and the right-most sub-tree has only 1 node, where $x = |\theta|$, i.e. x is the total number of objects. Thus, a naive attempt to parallelize the exploration by distributing the first-level child nodes among the worker machines will cause severe load imbalance. Further, due to pruning of the zero-contributing sets during the execution, the compute load among the workers will vary during run-time. In the next section, we derive the primary requirements of a framework to implement our strategy of parallelizing skyline computation across different workers.

3.4 Selecting the right framework for scalable computation of skyline probabilities

- (1) The approach presented in the previous section can be modeled as an instance of combinatorial search, viz. enumerating the exponential search space while pruning the absorbed nodes. Thus, a suitable framework for parallelization should provide APIs (application programming interfaces) for expressing combinatorial search. The popular data analytics

frameworks like Spark¹, and Hadoop MapReduce² are not suited for this problem as it is not data parallel in nature.

- (2) Given the highly compute-intensive nature of this problem, the framework should support distributed computation (DC). Only then, it can take advantage of the parallelism available within and across machines.
- (3) Given the highly irregular nature of the computation tree, the framework should provide dynamic load balancing, i.e. it must distribute the computation tree evenly among the workers at runtime.
- (4) As skyline computation tasks are long-running, workers can fail during execution. Hence, the framework should support fault-tolerance (FT).
- (5) A non-essential but desirable requirement is the support for elastic scaling (ES).

Based on the above requirements, we surveyed several open-source combinatorial search frameworks and narrowed down to 4 of them, viz. YewPar [1], HOPE [16], Equilibrium [8] and DiGTreeS [7]. Their features are shown in Table 3. YewPar and Equilibrium do

Table 3: Feature comparison between frameworks

Name	Easy-to-use API	DC	FT	ES
YewPar	Y	N	N	N
Equilibrium	N	Y	N	Y
HOPE	Y	Y	Y	N
DiGTreeS	Y	Y	Y	Y

not provide fault-tolerance whereas HOPE does not support elastic scaling. Hence, we chose DiGTreeS to implement our algorithm for skyline probability.

3.5 Overview of DiGTreeS

DiGTreeS [7] has an event-driven architecture; task assignment, task completion, request for donor, worker failure, scale-in, scale-out, etc. are considered as events in the system. DiGTreeS internally uses Apache ZooKeeper [6] for event detection and coordination among the workers. For communication, DiGTreeS provides high throughput messaging implemented over Apache Kafka [10]. DiGTreeS provides fault-tolerance using uncoordinated checkpointing techniques. Each worker takes a checkpoint of its task queue periodically and stores them in HDFS. When a single worker or multiple workers fail, the recovery procedure of DiGTreeS rebuilds the system state from the checkpoints and ensures no loss of work. For expressing a combinatorial search application, DiGTreeS provides an API consisting of 3 abstract classes: Data, Node and NodeFactory. The Data class defines the immutable global data shared by all the tasks in the framework. It provides abstract methods for reading from the input file and storing them. The Node class represents a node in the exploration tree. It provides 3 abstract methods primarily, nextChild() where the user specifies the algorithm to generate the next child node; enumStep(Accumulator) to specify how the accumulator should be updated at each enumeration step and estimateWork() to provide an estimate of the work associated

¹<https://spark.apache.org/>

²<https://hadoop.apache.org/>

with that node (i.e. the work required for processing the entire subtree beneath that node). The last class NodeFactory provides abstract methods for instantiating the Node class object from the input data.

The concrete implementations of the abstract interfaces are required to be provided to the DiGTreeS framework by the user. The runtime engine of DiGTreeS internally invokes the user-specified nextChild() method to generate the next child in the search tree. Then it invokes the enumStep to compute the specified metric at that child node and updates the accumulator. At the end of the computation, the DiGTreeS framework aggregates the accumulator values from all the workers and returns the final value of the accumulator.

3.6 Computing skyline probability using DiGTreeS

Our concrete implementation of the Data interface, viz. D-SkyData includes a 2D array for storing the multi-dimensional dataset, the total number of objects, the number of dimensions, and the preference probability. The DiGTreeS framework creates an instance of the D-SkyData object at the beginning of the computation and shares a readable copy with all the workers.

Our concrete implementation of the Node interface is D-SkyNode and is shown in Algorithm 1. It uses an array called prefix to store the path from the root till that node and markers array to store the objects that need to be added to the current node to generate the subtree beneath. In the NEXTCHILD procedure lines 1-11, we

Algorithm 1 D-SkyNode

Require: prefix, markers

```

1: procedure NEXTCHILD
2:   for i = 0 to markers.size() - 1 do
3:     k1 = prefix + markers[i]
4:     for j = i + 1 to markers.size() - 1 do
5:       k2 = prefix + markers[j]
6:       if distinct(O; k2) = distinct(O; k1 ∪ k2) then
7:         Flag the  $j^{th}$  element from markers
8:       else if distinct(O; k1) = distinct(O; k1 ∪ k2)
9:         Flag the  $i^{th}$  element from markers
10:    remove flagged elements from the markers list
11:    newprefix = prefix + markers[0]
12:    newmarkers = markers - markers[0]    ▶ set difference
13:    return new D-SkyNode(newprefix, newmarkers)

```

generate the child nodes by adding an object from the markers to the current node one at a time. Next we evaluate which of the child nodes are absorbed according to Definition 1. These child nodes are flagged for removal and later removed from the markers in line 12. The first child's identifier is added to the prefix value stored at the current node to obtain the newPrefix value for the child node. The newPrefix and the updated markers is passed as an argument to the D-SkyNode constructor to return the next child node in the subtree beneath (line 15). In the enumStep method, we compute the probability of the event corresponding to the current node and

Table 4: Dataset characteristics

Dataset	x	d	NodesExpl.	Type	Seq.Ex.
Lenses	24	5	31	Real	<1s
Zoo	101	17	1	Real	<1s
Balance	625	5	2,59,151	Real	40s
10k-med.	10000	8	70,27,08,895	Syn.	3,595s
Higgs	10000	12	1,32,09,32,287	Real	10,908s
10k-high	10000	9	12,87,77,44,431	Syn.	82,653s
15k-high	15000	9	32,53,20,06,777	Syn.	2,03,834s

add its signed value to the accumulator. The DiGTreeS framework aggregates the event probability values from all the workers and returns the skyline probability of the object. To implement the abstract method, estimateWork, we use the worst-case estimate of the number of nodes in the subtree below the current node, which is $2^{\text{size}(\text{markers})}$. The DiGTreeS framework aggregates this work estimate over all the nodes present in the taskQueue of a worker to arrive at an estimate of the worker's compute load. DiGTreeS uses the estimate of the worker's compute load in its dynamic load balancing (DLB) algorithm. Whenever an idle worker requests for work, its request is satisfied by the most loaded worker in the system. It has been shown that this DLB strategy reduces the number of work transfers and is quite effective in balancing the load evenly among the workers.

Deployment details: We need to first set up Hadoop, ZooKeeper and Kafka cluster. Then we need to install DiGTreeS on all the machines. Finally, for executing DiGTreeS, we need to specify the IP addresses of the above clusters, the target machines to be used in our experiments and the input dataset. DiGTreeS then executes the program on the specified machines by distributing the computation, managing faults and returning the results.

4 Experimental evaluation

We first describe the distributed test-bed followed by the characteristics of both real-world and sythetic datasets. Next, we describe the speedup obtained by our distributed implementation (D_Sky) on these datasets. Then, we compare the performance of D_Sky with the state-of-the-art parallel implementation P_Sky w.r.t. execution time and load distribution among the workers. Then, we show the results obtained with elastic scaling. Finally, we study resource usage of our implementation w.r.t. CPU and memory utilization.

4.1 Test-bed

We use a test-bed consisting of 30 commodity machines, each having Intel i7-6700 CPU @ 3.40GHz, 8GB RAM with Ubuntu 20.04.4 LTS version. Additionally, we use a 3 machine cluster with the same configuration to run Hadoop v3.2.0 in pseudo-distributed mode, ZooKeeper v3.4.1 and Kafka v2.2.0 in cluster mode, respectively. On each machine, we run multiple instances of workers up to a maximum of 5 workers/machine. Each experiment is run 3-5 times and the average of those runs is reported³.

4.2 Dataset characteristics

We used real-world as well as synthetic datasets for our experiments. The real-world datasets include Lenses, Zoo, Balance and Higgs from the UCI ML repository⁴. Among them, the first 3 datasets are also used in [13, 19]. These are small to moderate-sized datasets with categorical attributes, and no missing values. In addition, we generated synthetic (Syn.) large datasets based on the procedure given in [13]. These synthetic datasets vary in size, with the number of objects ranging from 10000 to 15000 and the dimensions varying from 2 to 12. Each dimension is considered to be a categorical attribute with 4 values. Assuming mutual independence between the different dimensions⁵, each dimension is randomly generated with a uniform distribution. The data characteristics are shown in Table 4. x denotes the total number of objects in the dataset, d denotes the number of dimensions and ExploredNodes refers to the total number of nodes explored in the search tree. The largest dataset has 15000 objects, 9 categorical attributes and the total number of nodes explored is around 32 billion.

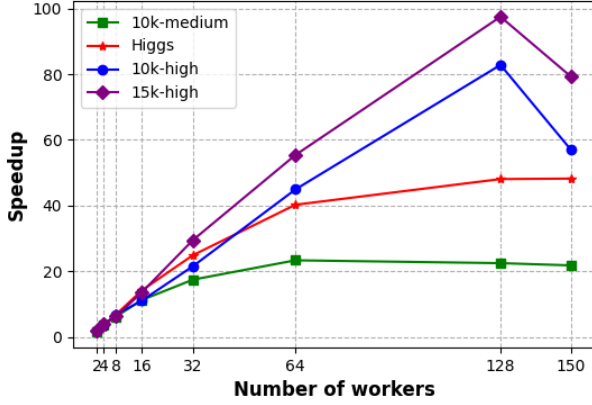


Figure 3: Speed up obtained by D_Sky

4.3 Speedup

Speedup is defined as: $\frac{\text{sequential execution time}}{\text{execution time with } w \text{ workers}}$. The sequential execution time for each dataset is shown in the last column of Table 4. For the first 2 real-world datasets, Lenses and Zoo, it is less than 1s while it is around 40s for Balance. The DiGTreeS framework is not suitable for parallelizing low latency executions as the initialization time for Hadoop, ZooKeeper, and Kafka itself is around 20s. Hence, we show the speedup results on the larger datasets (see Fig. 3). We vary the number of workers from 2 to 150, each time doubling the number of workers, i.e. the speedup results are calculated for 2 workers, 4 workers, 8 workers, ... 128 workers and then for 150 workers. We see that the speedup is maximum (around 98) for the largest dataset containing 15k objects. The number of workers used in the execution is 128 thereby giving a parallel execution efficiency⁶

³Being a controlled environment, the results show very less deviation.

⁴<https://archive.ics.uci.edu/datasets>

⁵a common assumption made by existing works

⁶ratio between speedup and the number of workers used

of 78%. As we increase the number of workers to 150 (i.e. beyond 128), the speedup decreases due to context-switching overheads between the workers running on the same machine. Recall that the processor used in our experiments is i7-6700 that provides 4 cores. When we run 150 workers on 30 machines, the number of workers per machine becomes 5. Hence, we notice a decline in the speedup. This behavior is consistent across all datasets. For the smallest dataset, 10K-medium, the speedup saturates around 23 with the corresponding execution time being 160s. This is due to the initialization overheads of DiGTreeS. For the larger datasets, viz. 10K-high and 15K-high, the execution time with 128 workers is greater than 1000s, thereby reducing the impact of the initialization overheads. Thus, our implementation provides higher speedup values on these larger datasets. The corresponding parallel efficiencies for these larger datasets is greater than 65%. From these results, we conclude that our distributed implementation of the skyline probability computation scales well.

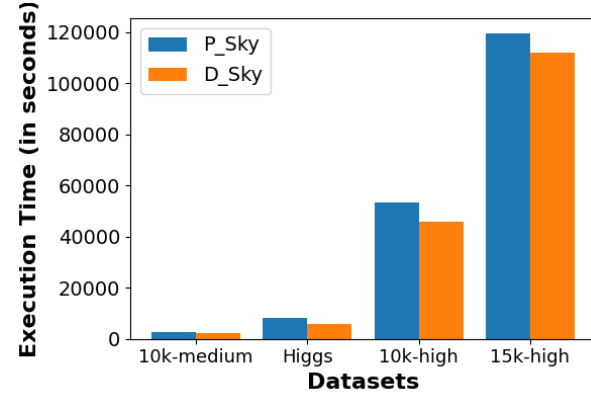
4.4 Comparison with state-of-the-art algorithm

Here, we compare the performance of our distributed implementation (D_Sky) with the state-of-the-art parallel implementation (P_Sky) [19]. P_Sky currently can only exploit parallelism available within a machine and not across different machines. Hence, we use a single machine as the test-bed and run multiple workers on it. As each machine in our test-bed has only 4 cores, we limit the performance comparison to 2 workers and 4 workers only (see Fig. 4a and 4b respectively). Clearly, D_Sky outperforms P_Sky on all datasets for both 2 workers and 4 workers. For 2 workers, the performance improvement of D_Sky over P_Sky calculated as $\frac{\text{execution time of P_Sky} - \text{execution time of D_Sky}}{\text{execution time of P_Sky}}$, ranges between 7% to 25%. With 4 workers, the performance improvement ranges between 25% to 51%. Thus, we see that the performance difference between D_Sky and P_Sky widens as the number of workers increases.

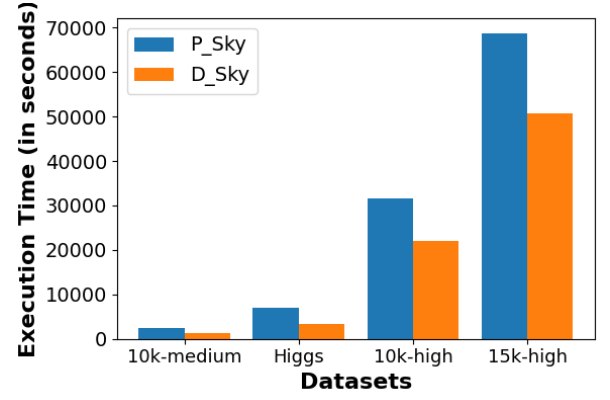
As discussed earlier, the search tree for computing the skyline probabilities is dynamically generated and is highly irregular due to pruning. P_Sky employs only static load-balancing thereby causing poor load distribution during runtime. To validate this, we measured the maximum deviation in the completion time of different workers, i.e. deviation = maximum completion time across all workers - minimum completion time across all workers (see Fig. 4c and 4d). The maximum value of deviation for D_Sky across all datasets is within 20s for 2 workers and 50s for 4 workers respectively, whereas for P_Sky, it oscillates between 1000s and 20,000s for both 2 and 4 workers. Thus, our hypothesis of P_Sky's poor performance due to uneven load during runtime is validated.

4.5 Elastic scaling

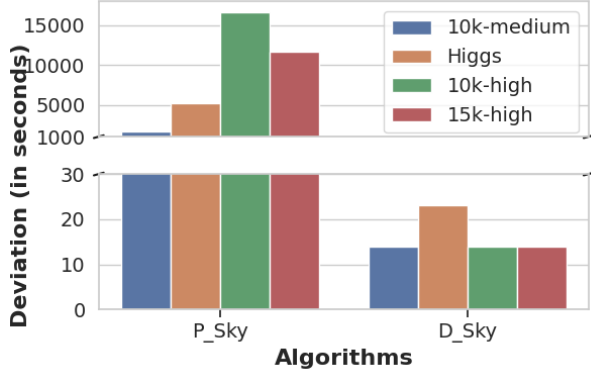
DiGTreeS allows the user to specify an efficiency range for the distributed computation. Its elastic scaling engine monitors the efficiency of each worker periodically. It scales-in (for increasing efficiency) or scales-out (for decreasing efficiency) to maintain the average efficiency over all workers during execution within the specified efficiency range (refer to [7] for more details). In our experiments, we specified the efficiency range as 85% to 95%. For comparing the performance of fixed worker execution with elastic



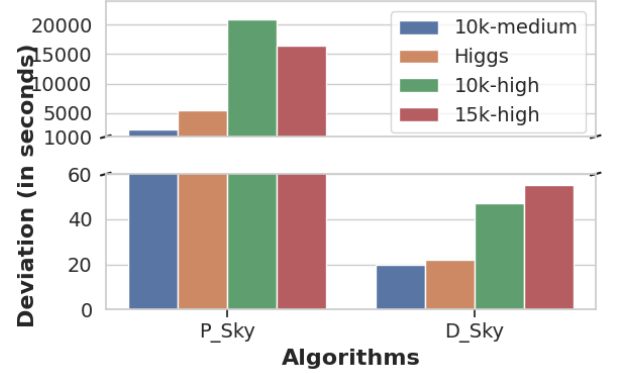
(a) With 2 workers



(b) With 4 workers



(c) With 2 workers



(d) With 4 workers

Figure 4: Comparison of D_Sky with state-of-the-art P_Sky[19] w.r.t. (a),(b) Execution Time (c), (d) Load imbalance (maximum deviation). D_Sky outperforms P_Sky in all the datasets. Performance of P_Sky suffers due to severe load-imbalance (see the large values for maximum deviation).

scaling, we used two metrics execution time (a.k.a. job completion time) and the cloud execution cost which is the sum of the running times of each worker during the entire job execution. For elastic scaling, a worker can be commissioned at any time and decommissioned at any time during execution. Thus, the running time of a worker during the job execution is the difference between the worker's decommissioning time and the worker's commissioning time. Figure 5 shows how the cloud execution cost (i.e. the sum of execution times of each worker) and the job completion time with 32 workers and 64 workers compares with elastic scaling for 10k-medium dataset and the Higgs dataset. In terms of execution time, using 64 workers yields the lowest execution time whereas in terms of cost, 32 workers yields the least cost. The performance with elastic scaling is in between. However, the difference in the cloud execution cost across 32 workers, elastic scaling and 64 workers is marginal. Thus, it appears that 64 workers yields the best performance. Similar results were obtained for the larger datasets.

To analyze the behavior of elastic scaling, we plotted the number of alive workers with time during a single run of elastic scaling

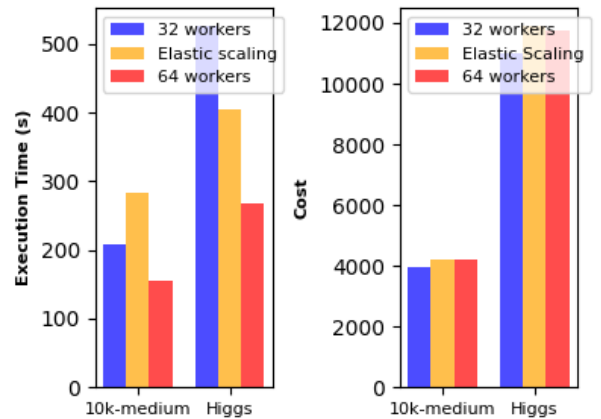


Figure 5: Elastic Scaling

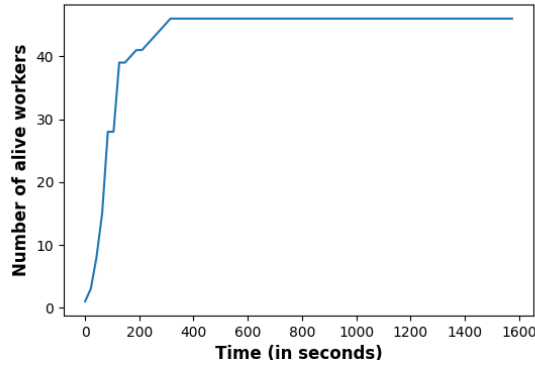


Figure 6: A single run with Elastic Scaling

(see Fig. 6) for the 10k-high dataset. The number of alive workers is 1 initially. DiGTreeS uses exponential scaling till 32 workers, and then switches to linear scaling. Thus, we observe that the number of workers quickly grows to 47 and remains stable until the end of the computation. There is no decommissioning of workers as the efficiency is maintained within the specified range. Hence, elastic scaling does not provide any major advantage to the distributed skyline probability computation except that it frees the user from calibrating the number of workers for execution.

4.6 Resource usage analysis of D_Sky

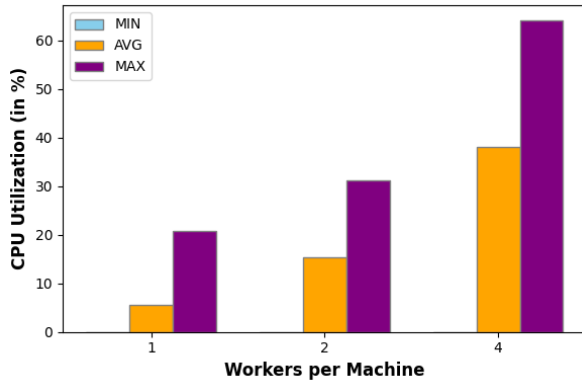


Figure 7: CPU Utilization

We measured the CPU usage and the memory usage of D_Sky by varying the number of workers per machine (see. Fig. 7 for CPU usage and Fig. 8 for memory usage). As the number of workers per machine is doubled, the CPU usage also doubles whereas the memory usage remains more or less stable. Thus, we see that computation of skyline probabilities is CPU-intensive. We also measured the communication overheads in D_Sky in terms of the network usage for work transfers. The network usage is minimal due to the efficient work distribution strategy adopted by DiGTreeS; each work transfer requires sending only a single node in the search tree to the requesting worker.

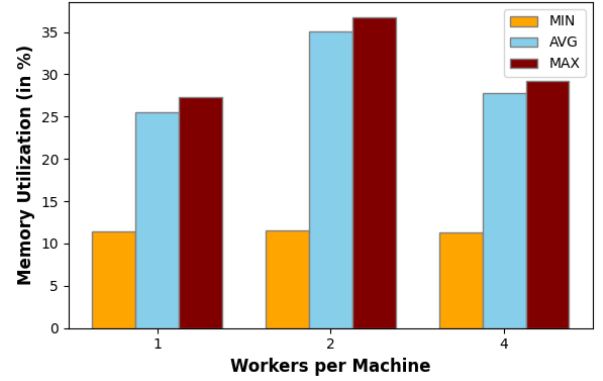


Figure 8: Memory Utilization

5 Conclusion

We were able to scale the computation of skyline probabilities to large datasets by reformulating the Usky-base algorithm as an enumeration problem within a combinatorial tree search paradigm and using DiGTreeS as the framework for realizing its implementation. The performance results highlight that D_Sky is able to achieve parallel execution efficiency of up to 78% on 128 workers for the largest dataset. Unlike other combinatorial search applications in [7], elastic scaling does not provide any major advantage to the distributed skyline probability computation except that it frees the user from calibrating the number of workers for execution. In future, we would like to extend D_Sky to compute skyline probabilities of all the objects at once.

References

- [1] Blair Archibald, Patrick Maier, Robert Stewart, and Phil Trinder. 2019. Implementing YewPar: A framework for parallel tree search. In *Euro-Par 2019: Parallel Processing: 25th International Conference on Parallel and Distributed Computing, Göttingen, Germany, August 26–30, 2019, Proceedings 25*. Springer, 184–196.
- [2] Ilaria Bartolini, Paolo Ciaccia, and Marco Patella. 2013. The Skyline of a Probabilistic Relation. *IEEE Transactions on Knowledge and Data Engineering* 25, 7 (2013), 1656–1669. <https://doi.org/10.1109/TKDE.2012.102>
- [3] Kenneth S. Bøgh, Sean Chester, and Ira Assent. 2015. Work-efficient parallel skyline computation for the GPU. *Proc. VLDB Endow.* 8, 9 (may 2015), 962–973. <https://doi.org/10.14778/2777598.2777605>
- [4] Xiaofeng Ding and Hai Jin. 2012. Efficient and Progressive Algorithms for Distributed Skyline Queries over Uncertain Data. *IEEE Transactions on Knowledge and Data Engineering* 24, 8 (2012), 1448–1462. <https://doi.org/10.1109/TKDE.2011.77>
- [5] Elaheh Gavagsaz. 2021. Parallel computation of probabilistic skyline queries using MapReduce. *J. Supercomput.* 77, 1 (Jan 2021), 418–444. <https://doi.org/10.1007/s11227-020-03279-x>
- [6] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. ZooKeeper: wait-free coordination for internet-scale systems. In *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference (Boston, MA) (USENIXATC'10)*. USENIX Association, USA, 11.
- [7] Md Arshad Jamal, Sriram Kailasam, Bhumanyu Goyal, and Varun Singh. 2024. DiGTreeS: a distributed resilient framework for generalized tree search. *J. Supercomput.* 80, 10 (Mar 2024), 15006–15037. <https://doi.org/10.1007/s11227-024-06017-9>
- [8] Stefan Kehrer and Wolfgang Blochinger. 2020. Equilibrium: an elasticity controller for parallel tree search in the cloud. *The Journal of Supercomputing* 76 (2020), 9211–9245.
- [9] Donald Kossmann, Frank Ramsak, and Steffen Rost. 2002. Chapter 25 - Shooting Stars in the Sky: An Online Algorithm for Skyline Queries. In *VLDB '02: Proceedings of the 28th International Conference on Very Large Databases*, Philip A. Bernstein, Yannis E. Ioannidis, Raghu Ramakrishnan, and Dimitris Papadias (Eds.). Morgan Kaufmann, San Francisco, 275–286. <https://doi.org/10.1016/B978->

- 155860869-6/50032-9
- [10] Jay Kreps, Neha Narkhede, Jun Rao, et al. 2011. Kafka: A distributed messaging system for log processing. In *Proc. of NetDB*. 1–7.
 - [11] Ken C. K. Lee, Baihua Zheng, Huajing Li, and Wang-Chien Lee. 2007. Approaching the skyline in Z order. In *Proceedings of the 33rd International Conference on Very Large Data Bases (Vienna, Austria) (VLDB '07)*. VLDB Endowment, 279–290.
 - [12] Jian Pei, Bin Jiang, Xuemin Lin, and Yidong Yuan. 2007. Probabilistic skylines on uncertain data. In *Proceedings of the 33rd International Conference on Very Large Data Bases (Vienna, Austria) (VLDB '07)*. VLDB Endowment, 15–26.
 - [13] Arun K Pujari, Venkateswara Kagita, Anubhuti Garg, and Vineet Padmanabhan. 2015. Efficient computation for probabilistic skyline over uncertain preferences. *Information Sciences* 324 (June 2015). <https://doi.org/10.1016/j.ins.2015.06.041>
 - [14] Lian Xiang Rai Niranjana. 2023. Distributed probabilistic top-k dominating queries over uncertain databases. *Knowledge and Information Systems* 65, 11 (2023), 4939–4965. <https://doi.org/10.1007/s10115-023-01917-3>
 - [15] Dimitris Sacharidis, Anastasios Arvanitis, and Timos Sellis. 2010. Probabilistic contextual skylines. In *IEEE 26th International Conference on Data Engineering (ICDE 2010)*. 273–284. <https://doi.org/10.1109/ICDE.2010.5447887>
 - [16] Masahiro Yasugi, Daisuke Muraoka, Tasuku Hiraishi, Seiji Umatani, and Kento Emoto. 2019. HOPE: a parallel execution model based on hierarchical omission. In *Proceedings of the 48th International Conference on Parallel Processing*. 1–11.
 - [17] P. Zhang, C. Zhou, P. Wang, B. J. Gao, X. Zhu, and L. Guo. 2015. E-Tree: An Efficient Indexing Structure for Ensemble Models on Data Streams. *IEEE Transactions on Knowledge and Data Engineering* 27, 02 (feb 2015), 461–474. <https://doi.org/10.1109/TKDE.2014.2298018>
 - [18] Qing Zhang, Pengjie Ye, Xuemin Lin, and Ying Zhang. 2013. Skyline probability over uncertain preferences. In *Proceedings of the 16th International Conference on Extending Database Technology (Genoa, Italy) (EDBT '13)*. Association for Computing Machinery, New York, NY, USA, 395–405. <https://doi.org/10.1145/2452376.2452423>
 - [19] Haoyang Zhu, Peidong Zhu, Xiaoyong Li, Qiang Liu, and Peng Xun. 2017. Parallelization of skyline probability computation over uncertain preferences. *Concurrency and Computation: Practice and Experience* 29, 18 (2017), e4201. <https://doi.org/10.1002/cpe.4201>